Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский политехнический университет»

Образовательная программа «Основы серверной веб-разработки»
Специализация: «Веб-технологии»
Направление подготовки: 09.03.01 «Информатика и вычислительная техника»

Отчет по курсовой работе
по дисциплине «Серверная веб-разработка»

Тема: «Разработка интернет-магазина техники на backend-фреймворке PHP (Laravel)»

Выполнил:
Студент группы 253-325
_____ Емельянов Н.С.
подпись, дата

Принял:
Старший преподаватель кафедры ИКТ
_____ Володина О.В.
подпись, дата

Москва, 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 4

Глава 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ 5

1.1 Анализ предметной области 5
1.2 Анализ существующих решений 5
1.3 Проектирование базы данных 6
1.4. Анализ функциональности проекта 7
1.5 Требования к системе 7
Глава 2. ПРАКТИЧЕСКАЯ ЧАСТЬ 8

2.1. Выбор технологий и архитектура приложения 8
2.2. Реализация базы данных и моделей 8
2.3. Реализация витрины магазина 9
2.4. Реализация корзины и оформления заказа 10
2.5. Реализация административной части 10
ЗАКЛЮЧЕНИЕ 11

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 12

ПРИЛОЖЕНИЕ А. Скриншоты работы приложения 13

ПРИЛОЖЕНИЕ Б. Ключевые фрагменты кода 18

ВВЕДЕНИЕ 4

Глава 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ 5

1.1 Анализ предметной области 5
1.2 Анализ существующих решений 5
1.3 Проектирование базы данных 6
1.4. Анализ функциональности проекта 7
1.5 Требования к системе 7
Глава 2. ПРАКТИЧЕСКАЯ ЧАСТЬ 8

2.1. Выбор технологий и архитектура приложения 8
2.2. Реализация базы данных и моделей 8
2.3. Реализация витрины магазина 9

2.4. Реализация корзины и оформления заказа 10
2.5. Реализация административной части 10
ЗАКЛЮЧЕНИЕ 11
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 12
ПРИЛОЖЕНИЕ А. Скриншоты работы приложения 13
ПРИЛОЖЕНИЕ Б. Ключевые фрагменты кода 18

ВВЕДЕНИЕ

Современный рынок компьютерной и цифровой техники развивается быстро: ассортимент обновляется несколько раз в год, а покупатель привык сравнивать характеристики, цены и наличие товара не выходя из дома. Для магазина техники наличие собственного сайта с актуальным каталогом перестало быть конкурентным преимуществом и стало базовым требованием: без него невозможно ни привлекать новых клиентов, ни автоматизировать обработку заказов.

Актуальность работы обусловлена тем, что небольшим магазинам и учебным проектам нужен инструмент, который позволяет быстро публиковать товары, управлять остатками и принимать заказы без привлечения дорогих готовых платформ. Backend-фреймворк позволяет собрать такое решение силами одного разработчика: он берёт на себя маршрутизацию, работу с базой данных, шаблонизацию и защиту от типовых атак.

Объект исследования — процесс продажи компьютерной техники через интернет.

Предмет исследования — веб-приложение интернет-магазина, реализованное на backend-фреймворке PHP.

Цель работы — разработать интернет-магазин техники на backend-фреймворке PHP (Laravel), обеспечивающий просмотр каталога, оформление заказа и управление ассортиментом через административную панель.

Для достижения цели необходимо решить следующие задачи:

Провести анализ предметной области и определить требования к системе.

Рассмотреть существующие решения и выделить их сильные и слабые стороны.

Спроектировать структуру базы данных и логическую модель приложения.

Реализовать витрину: главную страницу, каталог с фильтрами и карточку товара.

Реализовать корзину и оформление заказа с сохранением данных в базе.

Реализовать административную часть с авторизацией и управлением каталогом.

Проверить работоспособность сценариев и подготовить пояснительную записку.

Глава 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Интернет-магазин техники — это информационная система, в которой покупатель самостоятельно находит товар, изучает характеристики и оформляет заказ, а магазин получает готовую заявку с контактными данными. Основные сущности предметной области: товар, категория, заказ, позиция заказа и пользователь-администратор.

Ключевая особенность техники как товара — большое количество числовых характеристик (объём памяти, частота, мощность) и высокая цена. Поэтому каталог должен поддерживать фильтрацию по категории и цене, поиск по названию и бренду, а карточка товара — развёрнутое описание и остаток на складе. Покупателю важно видеть, есть ли товар в наличии, чтобы не оформлять заведомо невыполнимый заказ.

Целевая аудитория — студенты и сотрудники, подбирающие технику для учёбы и работы, а также небольшие организации, закупающие оборудование. Характер использования — преимущественно настольные компьютеры, поэтому интерфейс должен быть рассчитан на широкий экран, но корректно отображаться и на мобильных устройствах.

Бизнес-процесс магазина выглядит так: посетитель просматривает каталог, открывает карточку товара, добавляет позиции в корзину, указывает контактные данные и подтверждает заказ; администратор получает заказ, меняет его статус и корректирует остатки товаров. Все переходы этого процесса должны быть отражены в приложении.

1.2 Анализ существующих решений

Чтобы не проектировать интерфейс и структуру каталога с нуля, были рассмотрены четыре распространённых решения.

DNS и «М.Видео» — крупные маркетплейсы с развитой фильтрацией, сравнением товаров и программами лояльности. Их сильная сторона — объём данных

и рекомендации; слабая для учебного проекта — избыточность: поддерживать такую систему силами одного разработчика невозможно.

OpenCart и WordPress + WooCommerce — готовые движки для магазинов. Они дают административную панель почти «из коробки», но требуют изучения собственной архитектуры, а типовые темы сложно дорабатывать. Кроме того, в учебном задании важно продемонстрировать собственную реализацию, а не настройку готового продукта.

Самописный магазин на чистом PHP без фреймворка — минимальные зависимости, но высокая трудоёмкость: маршрутизацию, валидацию, защиту от CSRF и SQL-инъекций приходится реализовывать вручную, что увеличивает риск ошибок.

Вывод: для курсового проекта выбран backend-фреймворк Laravel. Он закрывает инфраструктурные задачи, оставляя разработчику прикладную логику, и при этом позволяет показать понимание MVC, ORM и миграций.

1.3 Проектирование базы данных

Модель данных построена по принципу «категория → товар → позиция заказа → заказ». Выделены пять таблиц.

Таблица categories хранит справочник категорий: идентификатор, название, slug (латинский идентификатор для адреса) и описание. Таблица products содержит товары: внешний ключ на категорию, название, slug, бренд, описание, цену, остаток на складе, путь к изображению и признак «хит продаж».

Заказ разделён на две таблицы. Orders фиксирует покупателя, контакты, адрес, комментарий, итоговую сумму и статус. Order_items хранит позиции заказа: ссылку на заказ, товар, название товара на момент покупки, цену и количество. Название и цена копируются в позицию специально: если товар изменится или будет удалён, история заказа останется корректной.

Связи: категория — один-ко-многим к товарам; заказ — один-ко-многим к позициям; позиция ссылается на товар через внешний ключ с обнулением при удалении товара. Для ускорения выборок добавлены индексы по паре «категория — цена» и по статусу заказа. Внешние ключи обеспечивают ссылочную целостность на уровне СУБД.

1.4. Анализ функциональности проекта

Функциональность системы разделена на две части: публичную витрину и административную панель.

Витрина: главная страница с категориями и подборками товаров; каталог с фильтрами по категории, цене и поиском; карточка товара с характеристиками и похожими товарами; корзина на основе сессии; оформление заказа с валидацией; страница подтверждения с номером заказа.

Административная панель: авторизация по e-mail и паролю; дашборд с ключевыми показателями; управление товарами (создание, редактирование, удаление, загрузка изображения, поиск); управление категориями; просмотр заказов и смена их статуса.

Отдельно выделены нефункциональные требования: защита форм от CSRF, проверка входных данных на сервере, параметризованные запросы к базе, разграничение доступа к административным разделам, адаптивная вёрстка.

1.5 Требования к системе

На основании анализа сформулированы требования к системе. Функциональные: просмотр каталога и карточки товара, фильтрация и поиск, добавление товара в корзину, оформление заказа, просмотр заказов и управление каталогом в админке.

Технические: серверный язык PHP версии 8.3 и выше, СУБД SQLite для разработки с возможностью перехода на MySQL, шаблонизация Blade, отсутствие обязательной сборки фронтенда, работа в браузерах Edge, Chrome, Firefox.

Ограничения: оплата онлайн не реализуется — заказ считается принятым после подтверждения формы; изображения товаров загружаются администратором; аутентификация предусмотрена только для администратора, регистрация покупателей не требуется.

Глава 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Выбор технологий и архитектура приложения

Приложение реализовано на Laravel 13 — одном из самых распространённых backend-фреймворков PHP. Выбор обоснован тремя причинами: фреймворк даёт готовую маршрутизацию, ORM и механизм миграций; он широко документирован; для него легко найти примеры решения типовых задач.

Стек: PHP 8.4, Laravel 13, шаблонизатор Blade, СУБД SQLite в режиме разработки с возможностью перехода на MySQL через переменные окружения. Фронтенд собран без сборщика: стили описаны в одном файле CSS, изображения товаров — векторные SVG. Такой подход упрощает развёртывание: для запуска не нужны Node.js и npm.

Архитектура соответствует паттерну MVC. Маршруты объявлены в routes/web.php и разделены на три группы: витрина, корзина и оформление, административная часть. Контроллеры принимают запрос, обращаются к моделям и возвращают представление. Модели Eloquent описывают сущности и связи, а бизнес-логика корзины вынесена в отдельный сервис App\Services\Cart, чтобы не дублировать её в контроллерах.

Каталог приложения разделён на слои: app/Http/Controllers — обработка запросов, app/Models — работа с данными, app/Services — прикладные сервисы, resources/views — шаблоны, database/migrations и seeders — схема и демо-данные, routes — маршруты. Такое разделение упрощает сопровождение и соответствует рекомендациям фреймворка.

2.2. Реализация базы данных и моделей

Схема базы данных создаётся миграциями — четырьмя файлами, которые выполняются командой php artisan migrate. Миграции описывают таблицы categories, products, orders и order_items, внешние ключи с каскадным удалением и индексы по паре «категория — цена» и по статусу заказа. Использование миграций позволяет развернуть одинаковую структуру на любой машине и откатить изменения.

Каждой таблице соответствует модель Eloquent. В модели Category объявлена связь hasMany с товарами, в Product — belongsTo с категорией, в Order — hasMany с позициями, в OrderItem — belongsTo с заказом и товаром. Типы полей приведены через свойство casts: цена хранится как decimal, остаток — как целое, признак «хит» — как boolean.

Фильтрация каталога реализована локальным скоупом scopeFiltered в модели Product. Скоуп последовательно применяет условия: категория по slug, поиск по названию, бренду и описанию, минимальная и максимальная цена, сортировка. Благодаря условиям when запрос собирается только из переданных параметров, а все значения подставляются через привязки — это исключает SQL-инъекции.

Демонстрационные данные создаёт DatabaseSeeder: администратор, пять категорий и восемнадцать товаров с ценами, остатками, описаниями и изображениями. Сидер использует updateOrCreate, поэтому повторный запуск не создаёт дубликатов.

2.3. Реализация витрины магазина

Главная страница формируется HomeController: он выбирает товары с признаком «хит», четыре последних товара и категории с количеством товаров. Подсчёт количества выполняется методом withCount, поэтому не требует отдельного запроса на каждую категорию.

Каталог реализован в CatalogController: параметры фильтра читаются из GET-запроса, применяется скоуп filtered, результат разбивается на страницы по девять товаров. Метод withQueryString сохраняет параметры фильтра в ссылках пагинации, поэтому переход между страницами не сбрасывает выбранную категорию и сортировку.

Карточка товара использует привязку модели по slug: маршрут /product/{product:slug} автоматически находит товар и возвращает 404, если его нет. Контроллер дополнительно загружает категорию и четыре похожих товара из той же категории — это динамический блок, зависящий от текущей записи.

Шаблоны Blade разделены на layout, страницы и переиспользуемые части. Карточка товара вынесена в partials/product-card и подключается на главной, в каталоге и в блоке похожих товаров; пагинация оформлена отдельным шаблоном.

Layout содержит шапку с категориями и счётчиком корзины, а также вывод flash-сообщений.

Вёрстка построена на CSS Grid и Flexbox, без внешних библиотек. Предусмотрены адаптивные состояния: на экранах уже 900 пикселей фильтры и карточка товара перестраиваются в одну колонку, меню переносится на новую строку.

2.4. Реализация корзины и оформления заказа

Корзина хранится в сессии в виде массива «идентификатор товара — количество». Сервис Cart инкапсулирует операции добавления, изменения количества, удаления и очистки, а также вычисление позиций и итоговой суммы. Позиции подгружаются одним запросом whereIn, что исключает проблему N+1.

Добавление товара выполняется POST-запросом на /cart/add/{product}. Контроллер проверяет остаток на складе и не позволяет добавить больше, чем есть в наличии. Изменение количества и удаление выполняются методами PATCH и DELETE с обязательным CSRF-токеном, поэтому изменить корзину со стороннего сайта нельзя.

Оформление заказа начинается с формы: имя, e-mail, телефон, адрес и комментарий. Данные проверяются валидатором Laravel — обязательность полей, формат e-mail, максимальные длины. Затем выполняется транзакция: создаётся заказ, для каждой позиции сохраняется снимок названия и цены, остаток товара уменьшается, корзина очищается. Транзакция гарантирует, что при ошибке на любом шаге база останется в согласованном состоянии.

После успешного оформления пользователь перенаправляется на страницу заказа, где видит номер, состав, сумму и статус. Перенаправление после POST защищает от повторной отправки формы при обновлении страницы.

2.5. Реализация административной части

Доступ в административную часть защищён middleware auth. Неавторизованный пользователь перенаправляется на /admin/login — адрес задан в конфигурации middleware, поэтому стандартный редирект на /login не используется. Вход выполняется методом Auth::attempt с проверкой пароля по хешу; после

успешной проверки идентификатор сессии перегенерируется — это защита от фиксации сессии.

Дашборд собирает агрегированные показатели: количество товаров, категорий и заказов, число новых заказов, сумму оплаченных заказов, список последних заказов и товары с остатком меньше пяти. Все значения получаются отдельными запросами с агрегатными функциями count и sum.

Управление товарами реализовано ресурсным контроллером: список с поиском, создание, редактирование и удаление. Изображение загружается через форму, сохраняется на диск public в каталог products и удаляется вместе с товаром. Slug генерируется из названия автоматически, но может быть задан вручную; уникальность проверяется правилом Rule::unique с игнорированием текущей записи при редактировании.

Управление категориями устроено аналогично, но содержит дополнительное правило: категорию, в которой есть товары, удалить нельзя — контроллер возвращает сообщение об ошибке. Заказы доступны только для просмотра и смены статуса: новый, оплачен, отправлен, выполнен, отменён. Статусы вынесены в константу модели Order, что позволяет использовать их и в формах, и в бейджах списка.

Интерфейс административной части построен на отдельном layout: тёмный сайдбар с группами пунктов, липкая шапка с хлебными крошками, карточки статистики, панели и таблицы со «зеброй» и закреплённой строкой заголовка. Стили адаптированы под узкие экраны: на мобильных меню разворачивается в горизонтальную строку.

ЗАКЛЮЧЕНИЕ

В ходе работы разработан интернет-магазин техники на backend-фреймворке Laravel. Все задачи, поставленные во введении, выполнены.

Проведён анализ предметной области: определены участники процесса, основные сущности и последовательность действий покупателя и администратора. Рассмотрены четыре существующих решения — от крупных маркетплейсов до готовых движков — и обоснован выбор собственной реализации на фреймворке.

Спроектирована база данных из пяти таблиц со связями «один ко многим» и внешними ключами. Реализованы миграции, модели Eloquent, демонстрационные данные.

Собрана витрина: главная страница с подборками, каталог с фильтрами и пагинацией, карточка товара с блоком похожих товаров. Реализованы корзина на сессии и оформление заказа с транзакционной записью в базу и списанием остатков.

Создана административная панель с авторизацией, дашбордом, управлением товарами и категориями, просмотром заказов и сменой статусов. Проверены сквозные сценарии: добавление товара в корзину, оформление заказа, вход в админку и работа всех её разделов.

Практическая значимость работы состоит в том, что приложение можно развернуть на любом хостинге с PHP и MySQL, а структура кода позволяет расширять функциональность. Направления развития: онлайн-оплата, личный кабинет покупателя, сравнение товаров, экспорт заказов и интеграция с службами доставки.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Документация Laravel 13 [Электронный ресурс]. — URL: https://laravel.com/docs (дата обращения: 10.11.2026).
2. Официальное руководство PHP [Электронный ресурс]. — URL: https://www.php.net/manual/ru/ (дата обращения: 10.11.2026).
3. Котеров Д. В. PHP 8 / Д. В. Котеров, И. В. Симдянов. — СПб.: БХВ-Петербург, 2021. — 992 с.
4. Зандстра М. PHP: объекты, шаблоны и методики программирования / М. Зандстра. — 6-е изд. — М.: Диалектика, 2021. — 736 с.

5. MDN Web Docs: HTML, CSS, JavaScript [Электронный ресурс]. — URL: https://developer.mozilla.org/ru/ (дата обращения: 10.11.2026).
6. SQLite Documentation [Электронный ресурс]. — URL: https://www.sqlite.org/docs.html (дата обращения: 10.11.2026).

MySQL 8.0 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/ (дата обращения: 10.11.2026).

Бэнкс А. Паттерны проектирования в PHP / А. Бэнкс, Д. Порчелло. — М.: ДМК Пресс, 2022. — 320 с.

ПРИЛОЖЕНИЕ А. Скриншоты работы приложения

Ниже приведены скриншоты основных страниц разработанного приложения: главная страница, каталог с фильтрами, карточка товара, корзина, оформление заказа, подтверждение заказа, дашборд административной панели и список заказов.

Скриншоты сняты в браузере Microsoft Edge при разрешении 1600×1000.

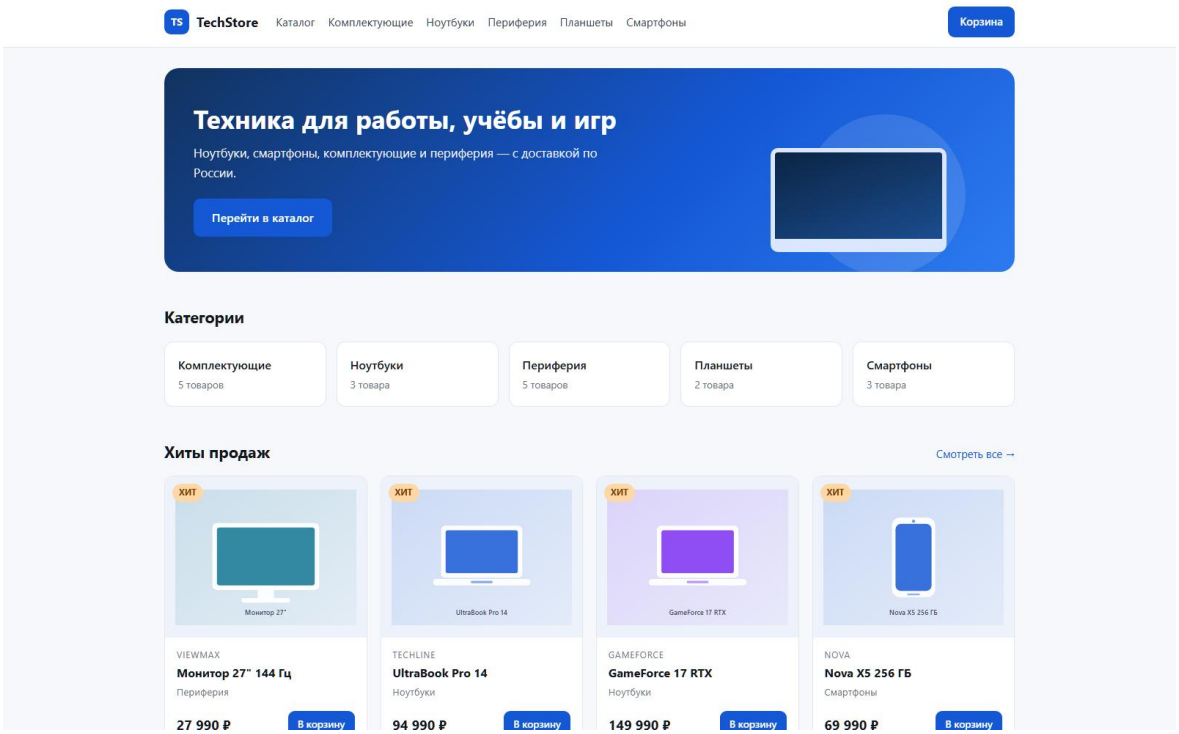


Рисунок А.1 — Главная страница магазина

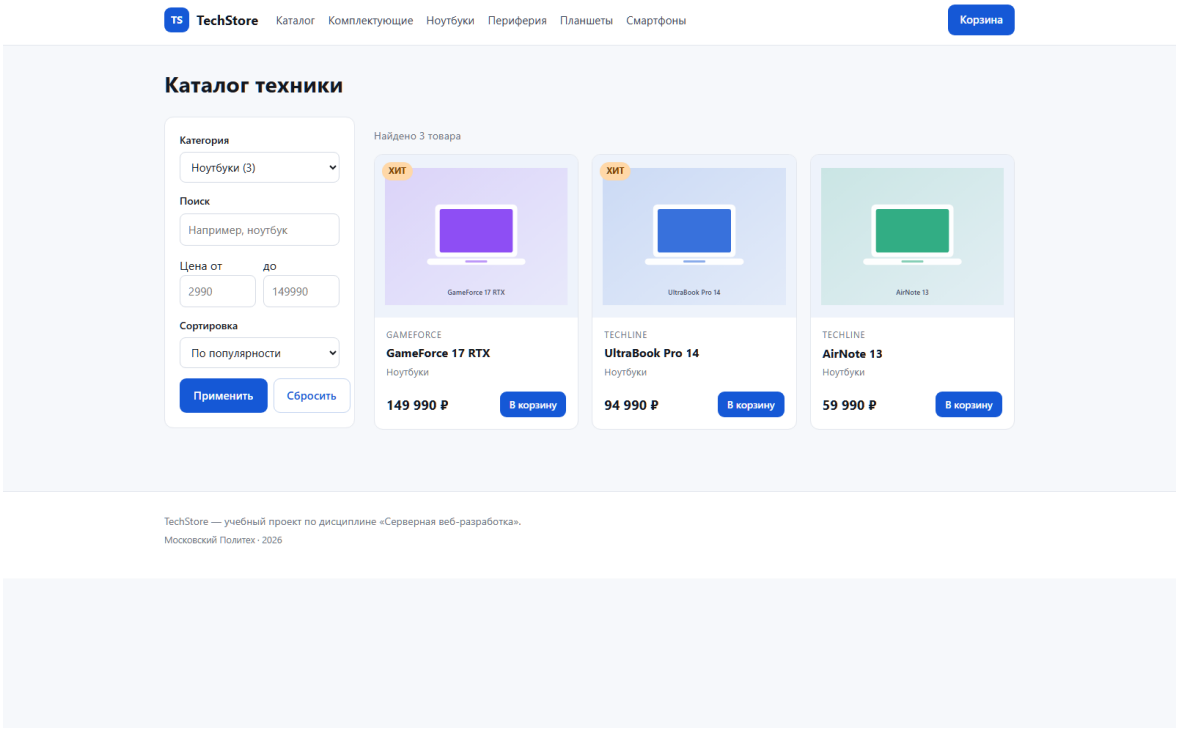


Рисунок А.2 — Каталог с фильтром по категории «Ноутбуки»

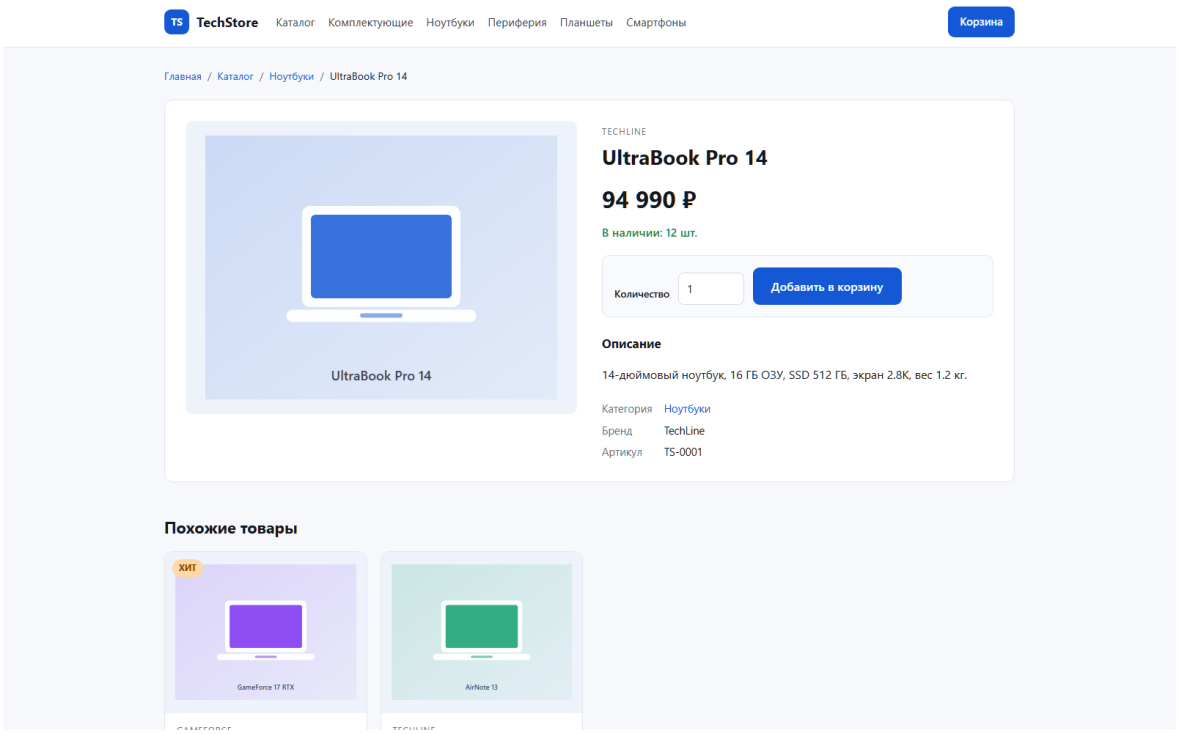


Рисунок А.3 — Карточка товара

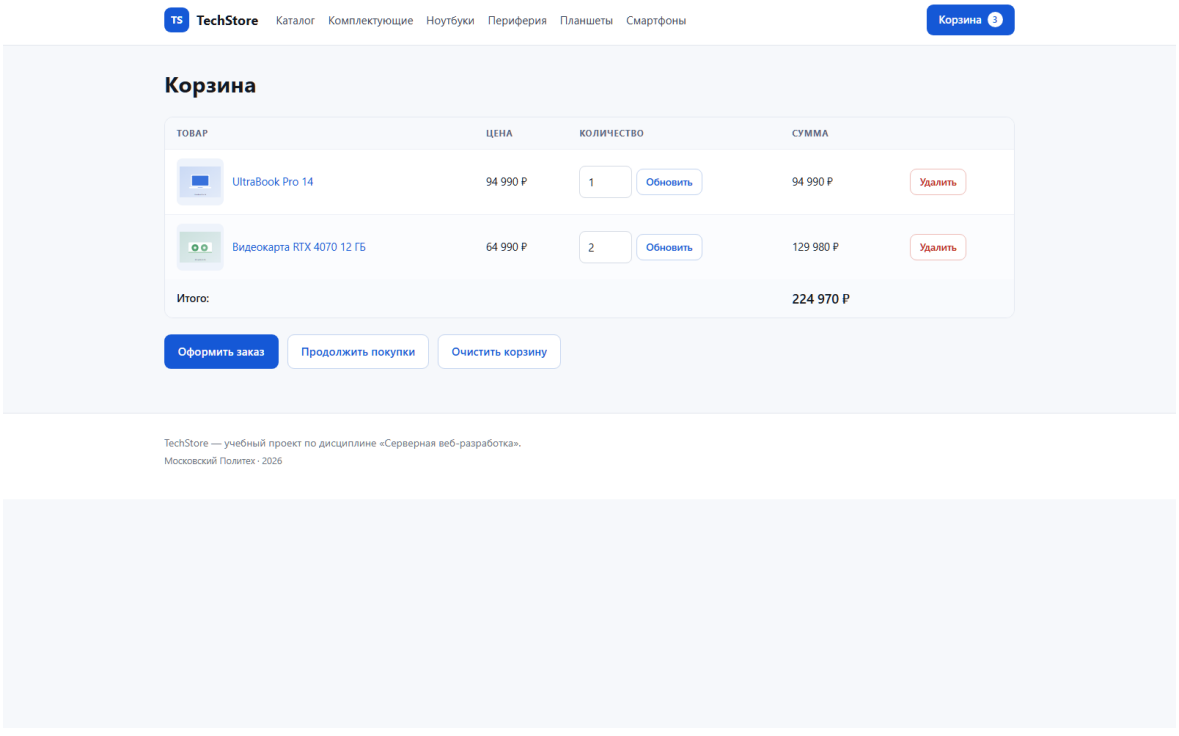


Рисунок А.4 — Корзина

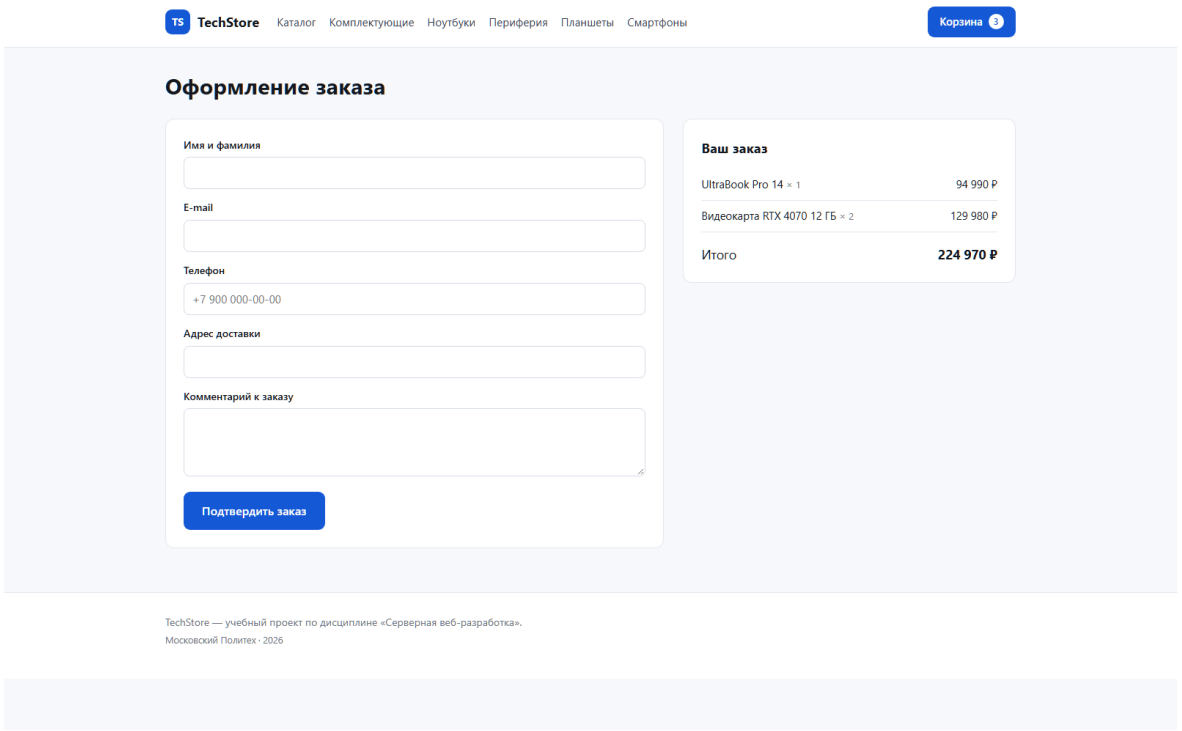


Рисунок А.5 — Оформление заказа

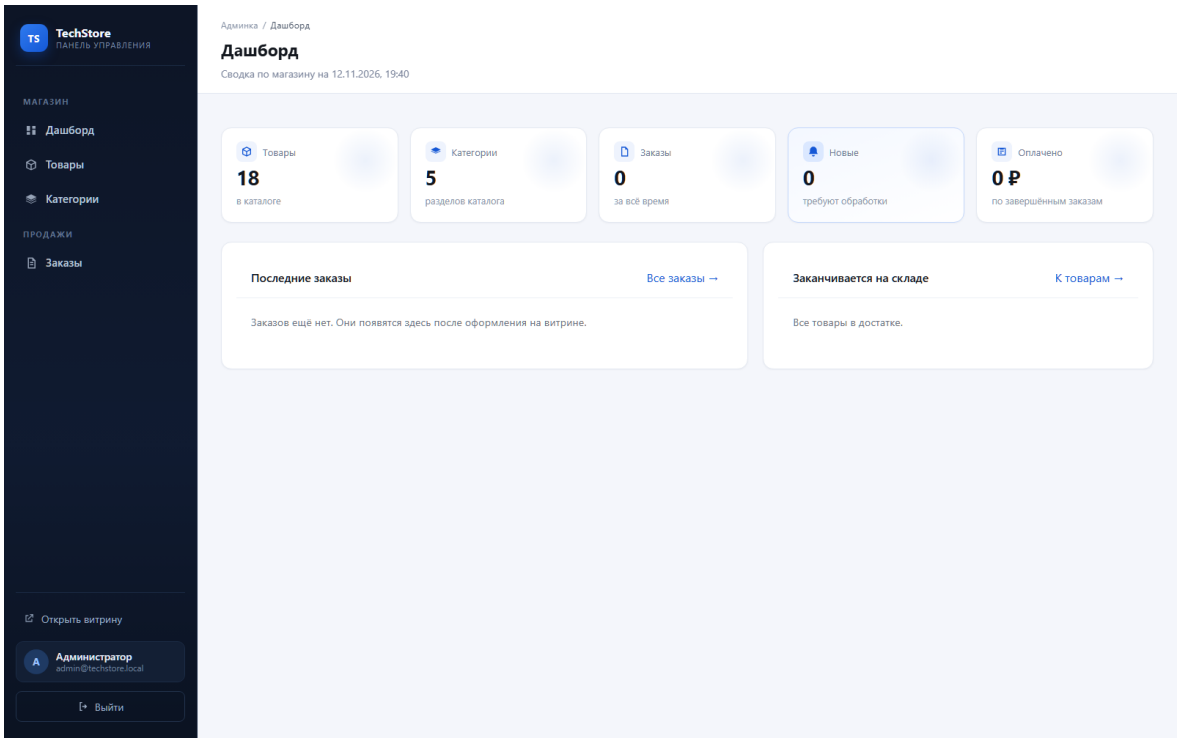


Рисунок А.6 — Дашборд административной панели

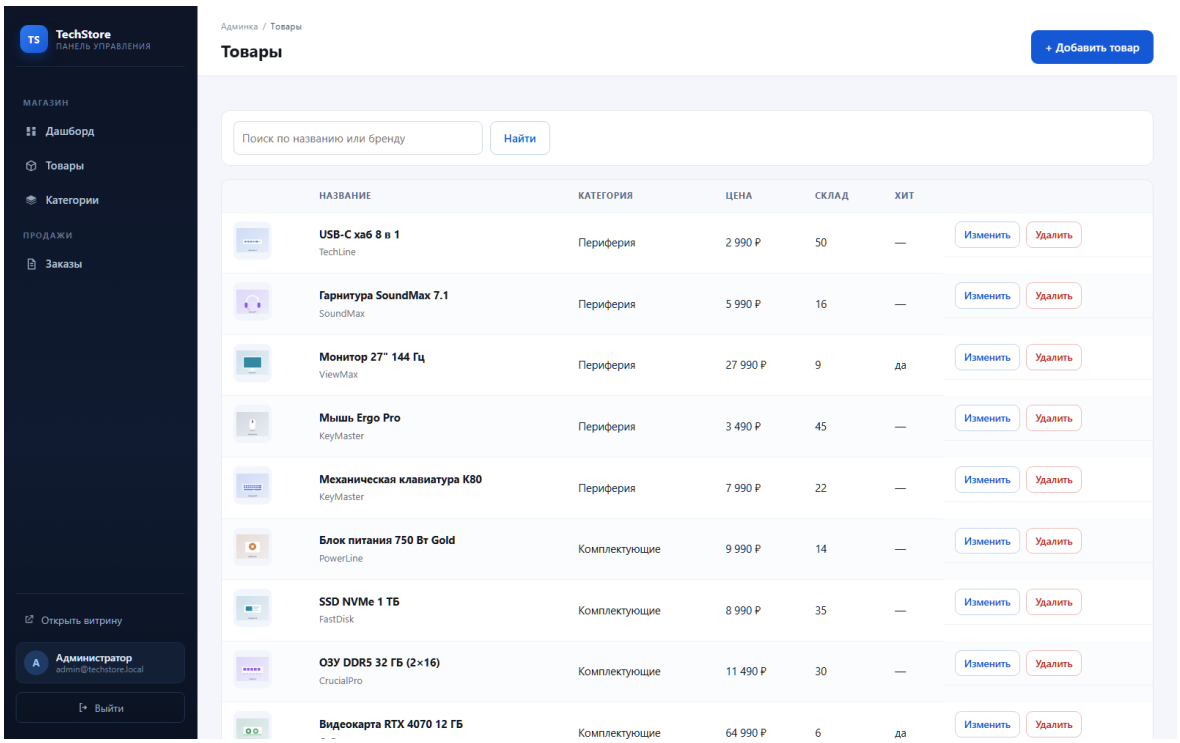


Рисунок А.7 — Управление товарами

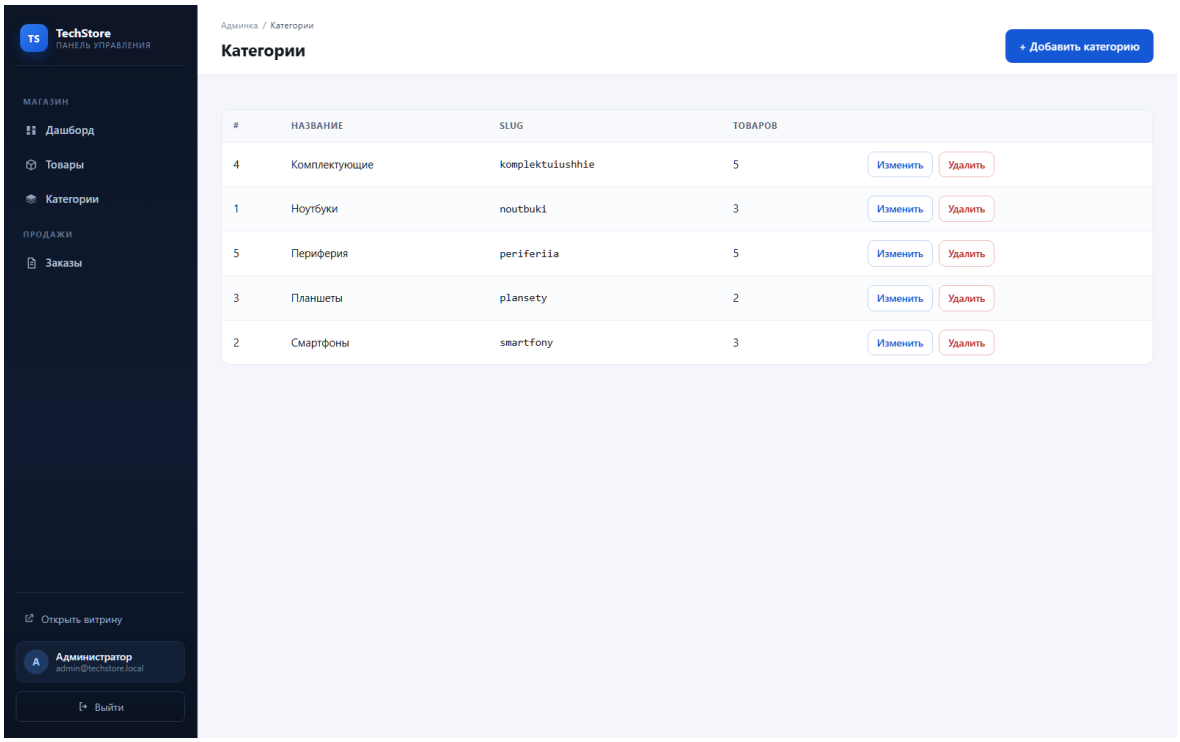


Рисунок А.8 — Управление категориями

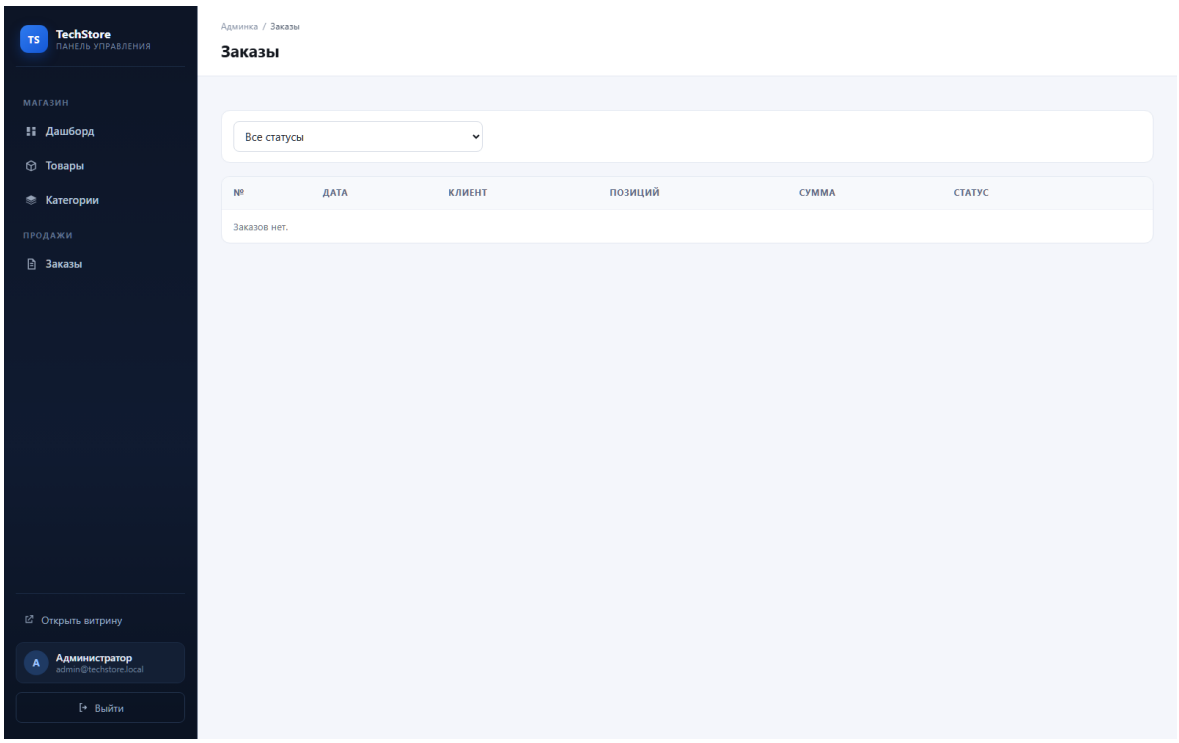


Рисунок А.9 — Список заказов

ПРИЛОЖЕНИЕ Б. Ключевые фрагменты кода

Б.1. Скоуп фильтрации каталога (app/Models/Product.php):

public function scopeFiltered(Builder $query, array $filters): Builder — последовательно применяет условия по категории, поисковому запросу, диапазону цен и сортировке.

Б.2. Оформление заказа (app/Http/Controllers/CheckoutController.php):

DB::transaction(...) — создание заказа, сохранение позиций со снимком цены и уменьшение остатков в одной транзакции.

Б.3. Авторизация администратора (app/Http/Controllers/Admin/AuthController.php):

Auth::attempt($credentials, $request->boolean('remember')) с последующей регенерацией идентификатора сессии.

Полные листинги находятся в репозитории проекта.